

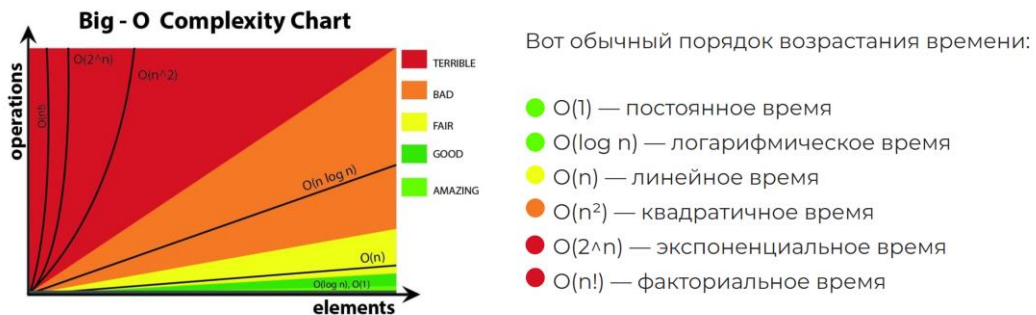
АЛГОРИТМЫ

Алгоритм — конечная совокупность точно заданных правил решения типовых задач или набор инструкций, описывающих порядок действий исполнителя **для решения задачи**.

Что такое Big O?

Big O — это мера эффективности «в ХУДШЕМ случае», т.е. верхняя граница того, сколько времени потребуется для выполнения задачи, или сколько памяти для этого необходимо.

Сравнение производительности разных коллекций по ссылке (аккуратно, можно зависнуть): <https://www.bigocheatsheet.com/>



Есть и другие «O»:

Обозначение	Граница	Рост
(Тета) Θ	Нижняя и верхняя границы, точная оценка	Равно
(О - большое) O	Верхняя граница, точная оценка неизвестна	Меньше или равно
(о - малое) o	Верхняя граница, не точная оценка	Меньше
(Омега - большое) Ω	Нижняя граница, точная оценка неизвестна	Больше или равно
(Омега - малое) ω	Нижняя граница, не точная оценка	Больше

О большое — **верхняя граница**, в то время как **Омега большое** — **нижняя граница**.

Тета Θ требует как **О большое**, так и **Омега большое**, поэтому она является **точной оценкой** (она должна быть ограничена как сверху, так и снизу).

К примеру, алгоритм, требующий **Омега Ω** ($n \log n$) требует не менее $n \log n$ времени, но верхняя граница не известна. Алгоритм требующий **Тета Θ** ($n \log n$) предпочтительнее потому, что он требует не менее $n \log n$ ($\Omega(n \log n)$) и не более чем $n \log n$ ($O(n \log n)$).

$f(x) = \Theta(g(n))$ означает, что f растет так же, как и g когда n стремится к бесконечности. То есть скорость роста $f(x)$ асимптотически пропорциональна скорости роста $g(n)$.

$f(x) = O(g(n))$ Здесь темпы роста не быстрее, чем $g(n)$.

О большое является наиболее полезной, поскольку представляет **НАИХУДШИЙ** случай, который пытаемся улучшить.

Короче говоря, если алгоритм имеет сложность __ тогда его эффективность __

Алгоритм	Эффективность
$o(n)$	$< n$
$O(n)$	$\leq n$
$\Theta(n)$	$= n$
$\Omega(n)$	$\geq n$
$\omega(n)$	$> n$

Какого типа задачи решаем при работе с коллекциями (основные операции)?

- Получение элемента по индексу
- Получение элемента по значению
- Добавление элемента в конец, в середину
- Удаление элемента с конца, середины

интерфейс
класс

Розовый – справедливо, оправдано: $O(n)$ или линейное время (n - это количество эл. коллекции)
Желтый – хорошо $O(\log n)$ - деревья
Зелёный – превосходно! $O(1)$ или константное время

структура данных, особенность			Временная сложность BigO							
			Среднее				Худшее			
			Индекс	Поиск	Вставка	Удаление	Индекс	Поиск	Вставка	Удаление
Iterable										
Collection										
List	упорядоченный список с дубликатами									
	ArrayList	динамический массив объектов	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
FIFO LIFO	LinkedList	дву-одно-связанный список	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$
	Vector	синхронизированный, устарел	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
LIFO	Stack									
Set	НЕ упорядоченное множество уникальных									
	HashSet	на основе хэш-таблицы	n/a	$O(1)$	$O(1)$	$O(1)$	n/a	$O(n)$	$O(n)$	$O(n)$
	LinkedHashSet	на основе связанного списка	n/a	$O(1)$	$O(1)$	$O(1)$	n/a	$O(n)$	$O(n)$	$O(n)$
	SortedSet	упорядоченный по возрастанию								
	NavigableSet	добавлена навигация								
	TreeSet	дерево	n/a	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	n/a	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Queue	FIFO очередь	вход - хвост, выход - голова								
FIFO	PriorityQueue	Очередь, выход по приоритету								
FIFO LIFO	Deque	LIFO/FIFO очередь двусторонняя								
	ArrayDeque	имеет реализацию кольцевой буфер								
ОТДЕЛЬНАЯ ВЕТВЬ, не наследуется от Collection										
Map<K, V>	карта пар ключ-значение (еще говорят "словарь")									
	SortedMap									
	NavigableMap									
	AbstractMap class									
	HashMap	массив бакетов + LinkedList (1св) при перестроении КЧ дерево	n/a	$O(1)$	$O(1)$	$O(1)$	n/a	$O(n)$	$O(n)$	$O(n)$
	LinkedHashMap	массив бакетов + LinkedList (2св) поддержка двусвязного списка	n/a	$O(1)$	$O(1)$	$O(1)$	n/a	$O(n)$	$O(n)$	$O(n)$
	WeakHashMap	Слабые ключи, элемент удаляется GC								
	TreeMap	дерево с возможностью навигации	n/a	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	n/a	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$

Шпаргалка тут → <https://habr.com/ru/post/188010/>

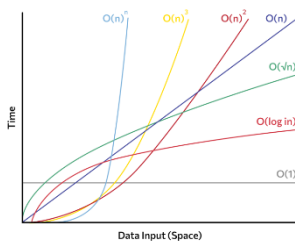
Сложность выполнения алгоритмов

по ссылке видео Алишева → <https://youtu.be/M3qhg2E9tPw>

BIG-O	Сложность	Пример
$O(1)$	Константная	Поиск по ключу в хэш-таблице; арифметическая операция с числом
$O(\log_2(n))$	Логарифмическая	Бинарный поиск, сложность, вставка сбалансированное бинарное дерево
$O(n)$	Линейная	Поиск перебором; среднее квадратическое отклонение
$O(n \cdot \log_2(n))$	Квазилинейное	Самые быстрые алгоритмы сортировки
$O(n^2)$	Квадратичная	Простые алгоритмы сортировки; перемножение n -значных чисел «столбиком»
$O(n^k)$	Полиномиальная	LU-разложение матрицы; мощность графа, базовые матем операции
$O(c^n)$	Экспоненциальная	Задача коммивояжёра (динамическое программирование)
$O(n!)$	Факториальная	Задача коммивояжёра перебором

- $O(1)$ означает, что алгоритм выполняется за фиксированное **константное** время. Это **самые эффективные** алгоритмы.
- $O(n)$ — это сложность **линейных** алгоритмов. n здесь и дальше обозначает **размер входных данных**: чем больше n , тем дольше выполняется алгоритм.
- $O(n^2)$ чем больше n , тем выше сложность. Но зависимость тут не линейная, а **квадратичная**, то есть скорость возрастает намного быстрее. Это **неэффективные** алгоритмы, например с **вложенными циклами**.
- $O(\log n)$ — более эффективный алгоритм. Скорость его выполнения рассчитывается **логарифмически**, то есть **зависит от логарифма n** .
- $O(\sqrt{n})$ — **квадратичный** алгоритм, скорость которого зависит от **квадратного корня из n** . Он менее эффективен, чем логарифмический, но **эффективнее линейного**.

Существуют также $O(n^3)$, $O(n^4)$ и другие малоэффективные алгоритмы с высокими степенями. Их сложность растёт очень быстро, и *их лучше не использовать*.



Графическое описание сложности

Лучше разобраться в сложности в O -нотации поможет график. Он показывает, как изменяется время выполнения алгоритма в зависимости от размера входных данных. Чем более пологую линию даёт график, тем эффективнее алгоритм.

Преимущества использования BigO

- Легче сравнивать алгоритмы
- Сама запись становится проще
- Не думаем о деталях

Минусы использования BigO

- Теряем информацию о константах

Время выполнения конкретных алгоритмов может отличаться, хотя сложность в терминах BigO одинаковая.

- BigO — это только асимптотическая оценка

Мы НЕ узнаем время выполнения алгоритма на конкретных аргументах. Мы узнаем то, как ведёт себя алгоритм при **ОЧЕНЬ БОЛЬШИХ** аргументах.

Алгоритм	Временная сложность		Пространственная сложность	
	Наилучший случай	Средний случай	Наихудший случай	Наихудший случай
Быстрая сортировка	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(\log(n))$
Сортировка слиянием	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Timsort	$O(n)$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Пирамидальная сортировка	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$
Сортировка пузырьком	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Сортировка вставками	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Сортировка выбором	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Сортировка Шелла	$O(n)$	$O((n \log(n))^2)$	$O((n \log(n))^2)$	$O(1)$
Блочная сортировка	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$
Поразрядная сортировка	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

Как происходит оценка асимптотической сложности алгоритмов?

Асимптотика – это поведение функции при стремлении аргумента к бесконечности.

Практически всегда существует несколько способов решения той или иной задачи: одни предполагают затратить много времени, другие ресурсов, а третьи помогают лишь приближённо найти решение.

	n	$n \log n$	n^2	2^n
$n = 20$	1 sec	1 sec	1 sec	1 sec
$n = 50$	1 sec	1 sec	1 sec	13 day
$n = 10^2$	1 sec	1 sec	1 sec	$4 \cdot 10^{13}$ year
$n = 10^6$	1 sec	1 sec	17 min	
$n = 10^9$	1 sec	30 sec	30 year	
max n	10^9	$10^{7.5}$	$10^{4.5}$	30

Теория алгоритмов – это наука, которая изучает общие характеристики алгоритмов и формальные модели их представления.

Важно также оценивать, как будет вести себя алгоритм при начальных значениях разного объёма и количества, какие ресурсы ему потребуются и сколько времени уйдёт на вывод конечного результата. Этим занимается раздел теории алгоритмов – теория асимптотического анализа алгоритмов.

Основными показателями асимптотической сложности алгоритма является время и объём памяти, необходимые для решения задачи в процессе работы программы.

Также при анализе сложности для класса задач определяется некоторое число, характеризующее некоторый объём данных – размер входа.

Итак, можем сделать вывод, что **сложность алгоритма – функция размера входа**.

Сложность алгоритма **может быть различной** при одном и том же размере входа, но различных входных данных.

Существуют понятия сложности в худшем, среднем или лучшем случае.

Обычно, оценивают сложность в ХУДШЕМ случае.

Временная сложность в худшем случае – функция размера входа, равная максимальному количеству операций, выполненных в ходе работы алгоритма при решении задачи данного размера.

Ёмкостная сложность в худшем случае – функция размера входа, равная максимальному количеству ячеек памяти, к которым было обращение при решении задач данного размера.

Порядок роста сложности или **асимптотическая сложность** описывает приблизительное поведение функции сложности алгоритма при большом размере входа. Из этого следует, что при оценке временной сложности нет необходимости рассматривать элементарные операции, **достаточно рассматривать ШАГИ алгоритма**.

Шаг алгоритма – совокупность **последовательно-расположенных** элементарных операций, время выполнения которых не зависит от размера входа, то есть ограничена сверху некоторой константой.

Буква O – это константа, которая означает **верхнее ограничение** сложности алгоритма.

Основные классы сложности, применяемые при анализе →

$f(n) = O(1)$ константа
 $f(n) = O(\log(n))$ логарифмический рост
 $f(n) = O(n)$ линейный рост
 $f(n) = O(n \cdot \log(n))$ квазилинейный рост
 $f(n) = O(n^m)$ полиномиальный рост
 $f(n) = O(2^n)$ экспоненциальный рост

ПИСАТЬ КОД – ЭТО ХОРОШО,
 ПИСАТЬ РАБОТАЮЩИЙ КОД – ЕЩЕ ЛУЧШЕ,
 НО ЛУЧШЕ ВСЕГО ПИСАТЬ **ОПТИМИЗИРОВАННЫЙ РАБОТАЮЩИЙ КОД** =)

Для различных значений n показаны значения соответствующих функций. Зелёным выделены значения меньше 10 миллионов — примерно столько элементарных операций может выполняться на современных компьютерах «быстро» для скорости реакции человека. Естественно, фактическая скорость зависит от констант, скорости компьютера и ещё многих параметров. Но для грубой оценки эта таблица подойдёт! Видна кардинальная разница между $O(n)$ и $O(n^2)$ и тем более $O(n^3)$.

А факториал с экспонентой применимы только для самых маленьких значений n . Это означает, что для обработки многомиллионных массивов данных, алгоритмы с оценкой $O(n^2)$ или медленнее уже не годятся.

n	$\log(n)$	$n \log(n)$	n^2	n^3	2^n	$n!$
10	1	10	100	1 000	1 024	3 628 800
100	2	200	10 000	1 000 000	1.E+30	9.E+157
1 000	3	3 000	1 000 000	1 000 000 000	1.E+301	очень много
10 000	4	40 000	100 000 000	1 000 000 000 000	очень много	очень много
100 000	5	500 000	10 000 000 000	1 000 000 000 000 000	очень много	очень много
1 000 000	6	6 000 000	1 000 000 000 000	1 000 000 000 000 000 000	очень много	очень много
10 000 000	7	70 000 000	100 000 000 000 000	1 000 000 000 000 000 000 000	очень много	очень много
100 000 000	8	800 000 000	10 000 000 000 000 000	1 000 000 000 000 000 000 000 000	очень много	очень много

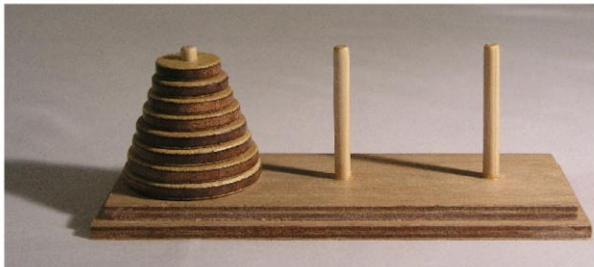
Что такое рекурсия?

Это функции, которые вызывают сами себя.

Рекурсия — определение, описание, изображение какого-либо объекта или процесса внутри самого этого объекта или процесса, то есть ситуация, когда объект является частью самого себя.



Термин «рекурсия» используется в различных специальных областях знаний — от лингвистики до логики, но наиболее широкое применение находит в математике и информатике.



Ханойская башня является одной из популярных головоломок XIX века.

Даны три стержня, на один из которых нанизаны восемь колец, причём кольца отличаются размером и лежат меньшее на большем.

Задача состоит в том, чтобы перенести пирамиду из восьми колец за наименьшее число ходов на другой стержень. За один раз разрешается переносить только одно кольцо, причём нельзя класть большее кольцо на меньшее.

Рекурсивно решаем задачу «перенести башню из $n-1$ диска на 2-й стержень». Затем переносим самый большой диск на 3-й стержень, и рекурсивно решаем задачу «перенести башню из $n-1$ диска на 3-й стержень».

Отсюда методом математической индукции заключаем, что минимальное число ходов, необходимое для решения головоломки, равно $2n - 1$, где n — число дисков.

В информатике задачи, основанные на легенде о Ханойской башне, часто рассматривают в качестве примера использования рекурсивных алгоритмов и преобразования их к не-рекурсивным.

Для того чтобы понять рекурсию, надо сначала понять рекурсию =)

Рекурсия состоит из **базового случая** и **шага рекурсии**. Базовый случай представляет собой самую простую задачу, которая решается за одну итерацию, например, `if(n == 0) return 1.`

В базовом случае обязательно присутствует **условие выхода из рекурсии**; **Смысл рекурсии** в движении от исходной задачи к базовому случаю, пошагово уменьшая размер исходной задачи на каждом шаге рекурсии.

После того, как будет найден базовый случай, срабатывает условие выхода из рекурсии, и стек рекурсивных вызовов разворачивается в обратном порядке, пересчитывая результат исходной задачи, который основан на результате, найденном в базовом случае.

Классическим примером рекурсии служит **вычисление факториала** числа. Факториал числа N - это произведение всех целых чисел от 1 до N . Например, факториал числа 3 равен $1 \times 2 \times 3$, т.е. 6. Ниже показано, как вычислить факториал, используя рекурсию.

```
public class Factorial {  
    // это рекурсивный метод  
    5 usages  
    int fact(int n) {  
        if (n == 1) {  
            return 1;  
        } else {  
            return fact(n - 1) * n;  
        }  
    }  
}
```

```
class Recursion {  
  
    public static void main(String args[]) {  
  
        Factorial fact = new Factorial();  
  
        System.out.println("Факториал 5 равен " + fact.fact(5));  
        System.out.println("Факториал 10 равен " + fact.fact(10));  
        System.out.println("Факториал 12 равен " + fact.fact(12));  
        System.out.println("Факториал 15 равен " + fact.fact(15));  
    }  
}
```

```
Факториал 5 равен 120  
Факториал 10 равен 3628800  
Факториал 12 равен 479001600  
Факториал 15 равен 2004310016  
Process finished with exit code 0
```

Сравните преимущества и недостатки итеративных и рекурсивных алгоритмов. С примерами.

Циклы дают лучшую производительность, чем рекурсивные вызовы, поскольку вызовы методов потребляют больше ресурсов, чем исполнение обычных операторов.

Циклы гарантируют отсутствие переполнения стека. В случае рекурсии стек вызовов разрастается, и его необходимо просматривать для получения конечного ответа.

При использовании головной рекурсии необходимо принимать во внимание размер стека.

Преимущества и недостатки использования рекурсии:

Рекурсию часто сравнивают с итерацией. Организация циклического процесса с помощью рекурсии имеет свои преимущества и недостатки.

ПРЕИМУЩЕСТВА рекурсии:

- естественность (натуральность) выражения сложных алгоритмов
- рекурсивный алгоритм более читабелен в сравнении с итерационным
- для многих распространенных задач рекурсию легче реализовать чем итерацию

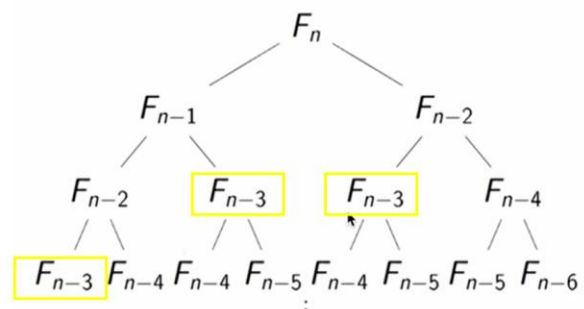
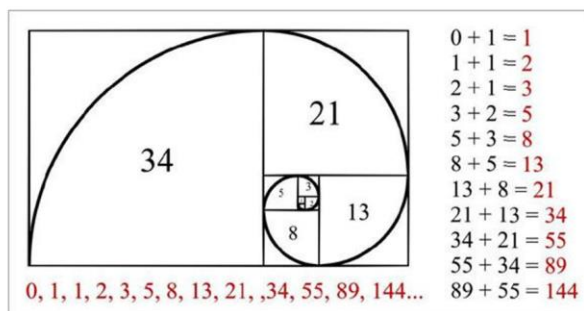
Рекурсия хорошо подходит для реализации алгоритмов обхода списков, деревьев, графов и т.д.

НЕДОСТАТКИ рекурсии:

По сравнению с итерацией **многократный вызов рекурсивной функции работает дольше**. Это связано с тем, что при вызове рекурсивного метода его **параметры копируются в стек**. Также запоминаются временные значения локальных внутренних переменных. При завершении вызова рекурсивной функции предыдущие значения параметров вытягиваются из стека, что приводит к лишним операциям.

Рекурсивное решение задачи «Числа Фибоначчи»

Очень медленное. Вычисление F_{100} займет 50.000 лет.



Итерационный алгоритм для такой же задачи работает БЫСТРЕЕ.

Для рекурсивного процесса **нужно больше памяти чем для итерации**. Это связано с тем, что при рекурсивном вызове нужно сохранять предыдущее значение внутренних переменных вызывающей функции, чтобы по завершении рекурсивного вызова восстановить ее выполнение.

Итак, если рекурсивная функция вызывается много раз, то это может привести к чрезмерно большому использованию памяти.

Итеративная

```
def factorial(number):
    product = 1
    for i in range(number):
        product = product * (i+1)
    return product
```

Handwritten notes: [0, 1, 2] under range, arrows showing loop iteration.

Рекурсивная

```
def factorial(number):
    if number <= 1: #base case
        return 1
    else:
        return number * factorial(number-1)
```

*Handwritten notes: factorial(3) = 3 * factorial(2) = 3 * 2 * factorial(1) = 3 * 2 * 1 = 6.*

Итеративная таблица:

i	product * (i+1)	Product
0	1 * (0+1)	1
1	1 * (1+1)	2
2	2 * (2+1)	6

Пример <https://youtu.be/OekWVc-3zfY>

Рекурсивный процесс — это процесс обработки данных с отложенными вычислениями.

Итеративный процесс — это процесс вычисления, когда состояние может быть описано фиксированным количеством значений (перебор).

Что такое жадные алгоритмы? Приведите пример.

<https://habr.com/ru/post/120343/>

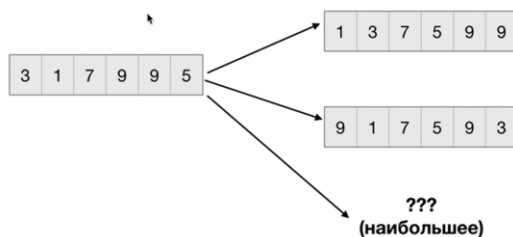
Выделяют три техники создания алгоритмов:

Смотрим Алишева <https://youtu.be/ccbj9NCGTDk>

- жадные алгоритмы (greedy algorithm)
- принцип «разделяй и властвуй» (divide and conquer)
- динамическое программирование

Жадный алгоритм — это алгоритм, который на каждом шаге делает **локально наилучший выбор** в надежде, что итоговое решение будет оптимальным (не всегда так).

Дан неупорядоченный список цифр от 0 до 9. Составить из этих цифр наибольшее число.



Жадная стратегия

На каждом шаге выбирать наибольшую цифру, присоединять ее к результирующему числу, удалять эту цифру из исходного массива

Это жадный алгоритм

```
package org.example;

import java.util.Arrays;

public class GreedyAlgorithms {
    public static void main(String[] args) {
        int[] digits = {3, 1, 7, 9, 9, 5};
        System.out.println(maxNumberFromDigits(digits));
    }
}
```

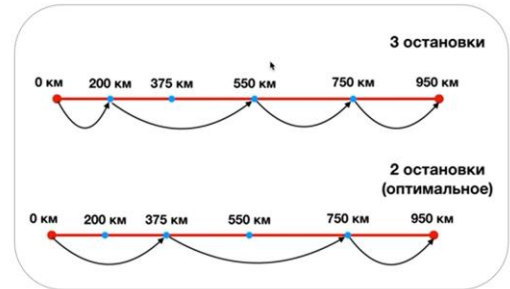
```
1 usage
public static String maxNumberFromDigits (int[] digits) {
    Arrays.sort(digits);
    String result = "";

    for (int i = digits.length - 1; i >= 0; i--) {
        result += digits[i];
    }

    return result;
}
```

Задача II

Вы едете на автомобиле.
Автомобиль может проехать на одной заправке 400 км.
На пути от точки А до точки В есть несколько заправок.
Надо проехать от точки А до точки В с минимальным количеством остановок на заправку



```
public class GreedyMinStops {
    public static void main(String[] args) {
        int [] stations = {0, 200, 375, 550, 750, 950};
        System.out.println(minStops(stations, capacity: 400));
    }

    1 usage
    public static int minStops(int[] stations, int capacity) {
        int result = 0;
        int currentStop = 0;

        while (currentStop < stations.length - 1) {
            int nextStop = currentStop;

```

```
// пока расстояние меньше, чем ёмкость бака, можем ехать.
// Если расстояние недостижимо, метод вернёт -1.

while (nextStop < stations.length - 1 &&
        stations[nextStop + 1] - stations[currentStop] <= capacity) {
    nextStop++;
}
if (currentStop == nextStop) {
    return -1; // вернуть, если след. точка недостижима
}
if (nextStop < stations.length - 1) {
    result++;
}
currentStop = nextStop;
}
return result;
}
```

Жадная стратегия

Заправляться на бензоколонке, которая дальше всех, но не дальше чем 400 км от текущей (оптимальный локальный выбор, который приведет к оптимальному глобальному выбору)

При построении Жадного алгоритма важно подумать, является ли ваше жадное локальное решение безопасным шагом!

Шаг называется безопасным, если он приводит к оптимальному решению.

Заправляться на самой дальней достижимой заправке - безопасный шаг (доказательство интуитивно понятно)

Иногда шаги бывают не безопасными

Пример небезопасного шага
(Жадный алгоритм не применим)

Проблема размена денег
(найти наименьшее количество монет, с помощью которых можно разменять данную сумму)

Есть монеты достоинством 1, 5, 10, 20, 25, 50 центов
Надо разменять 40 центов этими монетами

Жадная стратегия

На каждом шаге выбирать наибольшую по достоинству монету, не превышающую остаток

Ответ: (3 монеты)
40 = 25 + 10 + 5

Но можно: (2 монеты)
40 = 20 + 20

На каждом шаге выбирать наибольшую по достоинству монету - не безопасный ход. Жадный алгоритм не приводит к оптимальному решению

К примеру, алгоритм Дейкстры нахождения кратчайшего пути в графе вполне себе жадный, потому что мы на каждом шагу ищем вершину с наименьшим весом, в которой мы еще не бывали, после чего обновляем значения других вершин. При этом можно доказать, что кратчайшие пути, найденные в вершинах, являются оптимальными.

Пример формулировок задач (алгоритм Дейкстры):

Вариант 1. Дана сеть автомобильных дорог, соединяющих города Московской области. Некоторые дороги односторонние. Найти кратчайшие пути от города А до каждого города области (если двигаться можно только по дорогам).

Вариант 2. Имеется некоторое количество авиарейсов между городами мира, для каждого известна стоимость. Стоимость перелёта из А в В может быть не равна стоимости перелёта из В в А. Найти маршрут минимальной стоимости (возможно, с пересадками) от Копенгагена до Барнаула.

Решение:

Инициализация.

Метка самой вершины а полагается равной 0, метки остальных вершин — бесконечности. Это отражает то, что расстояния от а до других вершин пока неизвестны. Все вершины графа помечаются как не посещённые.

Шаг алгоритма.

Если все вершины посещены, алгоритм завершается. В противном случае, из ещё не посещённых вершин выбирается вершина u, имеющая минимальную метку.

Мы рассматриваем всевозможные маршруты, в которых u является предпоследним пунктом. Вершины, в которые ведут рёбра из u, назовём соседями этой вершины. Для каждого соседа вершины u, кроме отмеченных как посещённые, рассмотрим новую длину пути, равную сумме значений текущей метки u и длины ребра, соединяющего u с этим соседом.

Если полученное значение длины меньше значения метки соседа, заменим значение метки полученным значением длины. Рассмотрев всех соседей, пометим вершину u как посещённую и повторим шаг алгоритма.

```
class Dijkstra {
    double[] dist = new double[GV()];
    Edge[] pred = new Edge[GV()];
    public Dijkstra(WeightedDigraph G, int s) {
        boolean[] marked = new boolean[GV()];
        for (int v = 0; v < GV(); v++)
            dist[v] = Double.POSITIVE_INFINITY;
        MinPQplus<Double, Integer> pq;
        pq = new MinPQplus<Double, Integer>(); //Priority Queue
        dist[s] = 0.0;
        pq.put(dist[s], s);
        while (!pq.isEmpty()) {
            int v = pq.delMin();
            if (marked[v]) continue;
            marked[v] = true;
            for (Edge e (v)) {
                int w = e.to();
                if (dist[w] > dist[v] + e.weight()) {
                    dist[w] = dist[v] + e.weight();
                    pred[w] = e;
                    pq.insert(dist[w], w);
                }
            }
        }
    }
}
```

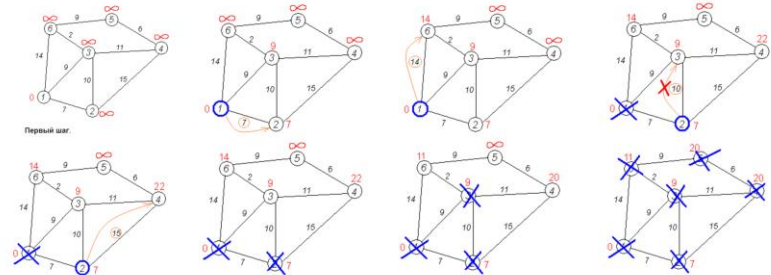
Рассмотрим выполнение алгоритма на примере графа, показанного на рисунке.

Пусть требуется найти кратчайшие расстояния от 1-й вершины до всех остальных.

Кружками обозначены вершины, линиями — пути между ними (рёбра графа).

В кружках обозначены номера вершин, над рёбрами обозначен их вес — длина пути.

Рядом с каждой вершиной красным обозначена метка: длина кратчайшего пути в эту вершину из вершины 1.

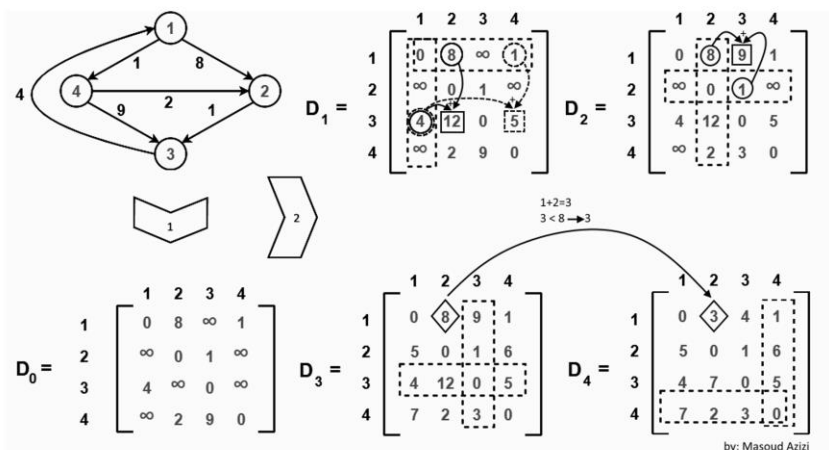


Результат работы алгоритма виден на последнем рисунке:

кратчайший путь от вершины 1 до 2-й составляет 7, до 3-й — 9, до 4-й — 20, до 5-й — 20, до 6-й — 11.

Напротив, алгоритм Флойда, который тоже ищет кратчайшие пути в графе (НО **между всеми вершинами**), **НЕ является примером жадного алгоритма**. Флойд демонстрирует другой метод — **метод динамического программирования**.

Задача: найти кратчайшие пути от любой вершины графа до любой другой вершины графа. <https://youtu.be/cgdyi19501Y>
Алгоритм Флойда позволяет найти кратчайшие пути между всеми парами вершин во взвешенном ориентированном графе.



Если граф взвешенный, значит у дуг есть ЗНАЧЕНИЯ.
В ориентированном графе не рёбра, а ДУГИ направления.

Задача о расписании

Пусть программисту-фрилансеру Васе дано n заданий. У каждого задания известен свой дедлайн, а также его стоимость (то есть если он не выполняет это задание, то он теряет столько-то денег). Вася настолько крут, что за один день может сделать одно задание. Выполнение задания можно начать с момента 0. **Нужно максимизировать прибыль.**

Классический пример применения жадина: Васе выгодно делать самые «дорогие задания», а наименее дорогие можно и не выполнять — тогда прибыль будет максимальна.

Возникает вопрос: каким образом распределить задания? Будем перебирать задания в порядке убывания стоимости и заполнять расписание следующим образом: если для заказа есть еще хотя бы одно свободное место в расписании раньше его дедлайна, то поставим его на самое последнее из таких мест, в противном случае в срок мы его не можем выполнить, значит поставим в конец из свободных мест.

```
//tasks - массив заданий
Arrays.sort(tasks); //сортируем по убыванию стоимости
TreeSet<Integer> time = new TreeSet<Integer>();
for (int i = 1; i <= n; ++i) {
    time.add(i);
}
int ans = 0;
for (int i = 0; i < n; ++i) {
    Integer tmp = time.floor(tasks[i].time);
    if (tmp == null) { // если нет свободного места в расписании, то в конец
        time.remove(time.last());
    } else { //иначе можно выполнить задание, добавляем в расписание
        time.remove(tmp);
        ans += tasks[i].cost;
    }
}
```

PS

Кстати, задачу можно решить и быстрее за $O(n)$. Кому не слабо? (Подсказка: нужно заменить TreeSet на другую структуру).

Задача о выборе заявок (пример жадины) https://youtu.be/yXYR_JuoidY

Есть довольно **мощный инструмент**, с помощью которого в большинстве случаев можно определить, даст ли жадина оптимальное решение. Называется **матроид**.

Как подсказывает википедия, **матроид** — это пара (X, I) , где X — конечное множество, называемое **носителем матроида**, а I — некоторое множество подмножеств X , называемое **семейством независимых множеств**. При этом должны выполняться следующие условия:

1. $\emptyset \in I$

Множество I непусто. Даже если исходное множество X было пусто — $X = \emptyset$, то I будет состоять из одного элемента — множества, содержащего пустое. $I = \{\{\emptyset\}\}$

2. $A \subset B, B \in I \Rightarrow A \in I$

Любое подмножество любого элемента множества I также будет элементом этого множества.

3. $|A| < |B| \Rightarrow \exists x \in B \setminus A, A \cup \{x\} \in I$

Если множества A и B принадлежат множеству I , а также известно, что размер A меньше B , то существует какой-нибудь элемент x из B , не принадлежащий A , такое что объединение x и A будет принадлежать множеству I . Это свойство является не совсем тривиальным, но чаще всего наиважнейшим из всех остальных.

Матроид называется **взвешенным**, если на множестве X существует аддитивная весовая функция w . Вес множества будет определяться как сумма весов его элементов.

[Расскажите про пузырьковую сортировку.](#)

Сортировка пузырьком / Bubble sort $O(n^2)$

Будем идти по массиву слева направо.

Если текущий элемент больше следующего, меняем их местами. Делаем так, пока массив не будет отсортирован. Заметим, что после первой итерации **самый большой элемент будет находиться в конце массива**, на правильном месте. После двух итераций на правильном месте будут стоять два наибольших элемента, и так далее.

Очевидно, не более чем после n итераций массив будет отсортирован. Таким образом, асимптотика в худшем и среднем случае – $O(n^2)$, **в лучшем случае – $O(n)$** .

```
public class BubbleSort {
    public static void main(String[] args) {
        int[] testData = {10, 4, 43, 5, 4, 67, 12, 0, 99, 19};
        System.out.println(Arrays.toString(bubbleSort(testData)));
    }
}
```

6	1	8	5	3
6	1	8	5	3
1	6	8	5	3
1	6	8	5	3
1	6	5	8	3
1	6	5	3	8

```
public static int[] bubbleSort(int[] array) {
    boolean sorted = false;
    int temp;
    while(!sorted) {
        sorted = true;
        for (int i = 0; i < array.length - 1; i++) {
            if (array[i] > array[i+1]) {
                temp = array[i];
                array[i] = array[i+1];
                array[i+1] = temp;
                sorted = false;
            }
        }
    }
    return array;
}
```

Расскажите про сортировку слиянием.

Сортировка слиянием / Merge sort $O(n \log n)$

Сортировка, основанная на **парадигме «разделяй и властвуй»**.

Разделим массив пополам, рекурсивно отсортируем части, после чего выполним процедуру слияния: **поддерживаем два указателя**, один на текущий элемент первой части, второй – на текущий элемент второй части. Из этих двух элементов выбираем минимальный, вставляем в ответ и сдвигаем указатель, соответствующий минимуму.

Слияние работает за $O(n)$, уровней всего $\log n$, поэтому асимптотика $O(n \log n)$.

Эффективно заранее создать временный массив и передать его в качестве аргумента функции. Эта сортировка рекурсивна, как и быстрая, а потому возможен переход на квадратичную при небольшом числе элементов.

На следующем рисунке показана процедура деления (разделения).

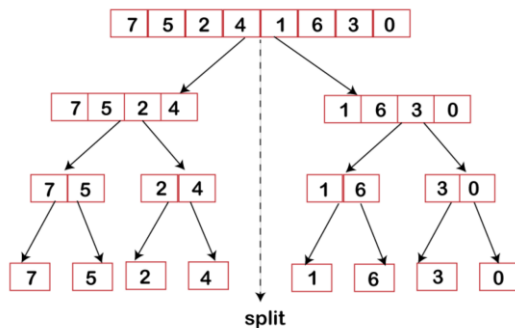


Figure 1: Merge Sort Divide Phase

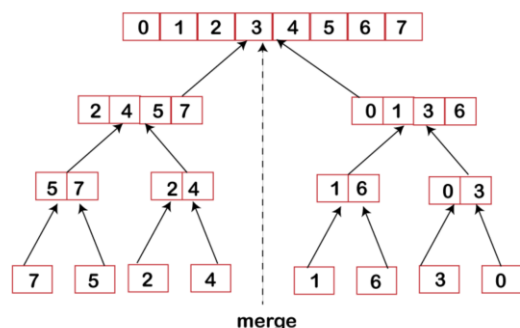


Figure 2: Merge Sort Combine Phase

```
// квазилинейная сложность: самый быстрый алгоритм
public class MergeSort {
    public static void main(String[] args) {
        int[] array1 = {8, 0, -3, 5, 6, 9, 8, -4, 2, -99, 43};
        int[] result = mergesort(array1);
        System.out.println(Arrays.toString(result));
    }

    1 usage
    public static int[] mergesort(int[] array1) {
        int[] buffer1 = Arrays.copyOf(array1, array1.length);
        int[] buffer2 = new int[array1.length];
        int[] result = mergesortInner(buffer1, buffer2, 0, array1.length);
        return result;
    }

    /**
     * @param buffer1 Массив для сортировки.
     * @param buffer2 Буфер. Размер должен быть равен размеру buffer1.
     * @param startIndex Начальный индекс в buffer1 для сортировки.
     * @param endIndex Конечный индекс в buffer1 для сортировки.
     * @return
     */
}
```

```
3 usages
public static int[] mergesortInner(int[] buffer1, int[] buffer2,
                                   int startIndex, int endIndex) {
    if (startIndex >= endIndex - 1) {
        return buffer1;
    }

    // уже отсортирован.
    int middle = startIndex + (endIndex - startIndex) / 2;
    int[] sorted1 = mergesortInner(buffer1, buffer2, startIndex, middle);
    int[] sorted2 = mergesortInner(buffer1, buffer2, middle, endIndex);

    // Слияние
    int index1 = startIndex;
    int index2 = middle;
    int destIndex = startIndex;
    int[] result = sorted1 == buffer1 ? buffer2 : buffer1;
    while (index1 < middle && index2 < endIndex) {
        result[destIndex++] = sorted1[index1] < sorted2[index2]
            ? sorted1[index1++] : sorted2[index2++];
    }
    while (index1 < middle) {
        result[destIndex++] = sorted1[index1++];
    }
    while (index2 < endIndex) {
        result[destIndex++] = sorted2[index2++];
    }
    return result;
}
```

Расскажите про быструю сортировку. Быстрая сортировка / Quicksort $O(n^2)$

Выберем некоторый **опорный элемент**. После этого перекинем все элементы, меньшие его, налево, а большие – направо. Рекурсивно вызовемся от каждой из частей.

В итоге получим отсортированный массив, так как каждый элемент меньше опорного стоял раньше каждого большего опорного. Асимптотика: $O(n \cdot \log n)$ в среднем и лучшем случае, $O(n^2)$. Наихудшая оценка достигается при неудачном выборе опорного элемента.

```
3 usages
public static void quickSort(int[] array, int low, int high) {
    if (array.length == 0)
        return;
    // завершить выполнение, если длина массива равна 0

    if (low >= high)
        return;
    // завершить выполнение если уже нечего делить

    // выбрать опорный элемент reference point
    int middle = low + (high - low) / 2;
    int referencePoint = array[middle];

    // разделить на подмассивы, который больше и меньше опорного элемента
    int i = low, j = high;
    while (i <= j) {
        while (array[i] < referencePoint) {
            i++;
        }

        while (array[j] > referencePoint) {
            j--;
        }

        if (i <= j) {
            // меняем местами
            int temp = array[i];
            array[i] = array[j];
            array[j] = temp;
            i++;
            j--;
        }
    }

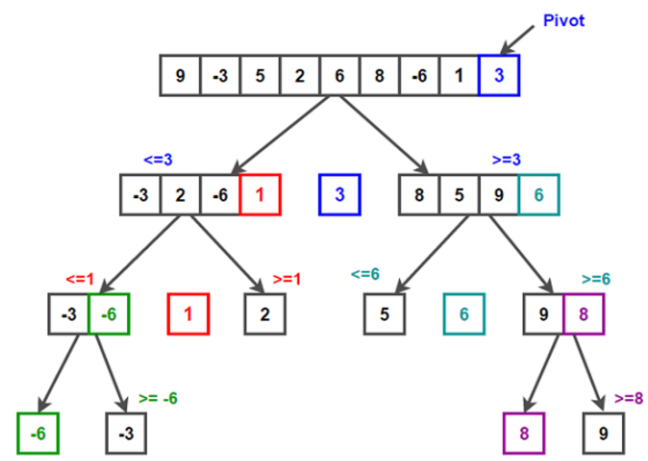
    // вызов рекурсии для сортировки левой и правой части
    if (low < j)
        quickSort(array, low, j);

    if (high > i)
        quickSort(array, i, high);
}
```

```
// логарифмическая сложность
public class QuickSort {
    public static void main(String[] args) {
        int[] testData = {8, 0, -3, 5, 6, 9, 8, -4, 2, -99, 43};

        int low = 0;
        int high = testData.length - 1;

        quickSort(testData, low, high);
        System.out.println(Arrays.toString(testData));
    }
}
```



Расскажите про бинарное дерево.

Двоичное/бинарное дерево, что это?

Ищем 16.

16 > 10? значит идём вправо

16 > 12? значит идём вправо

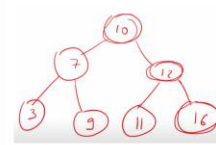
нашли!

БИНАРНЫЙ ПОИСК (два значения сравниваем)

Скорость будет $O(\log n)$

У двоичных деревьев может отсутствовать баланс.

Сбалансированное дерево (справа всегда БОЛЬШОЙ элемент, слева МЕНЬШОЙ)



Дерево переставит свои элементы (с помощью определённого алгоритма), оптимизируется, СБАЛАНСИРУЕТСЯ, чтобы облегчить поиск.

И получится

СБАЛАНСИРОВАННОЕ ДВОИЧНОЕ ДЕРЕВО

Зачем? позволяет быстро выполнять основные операции дерева поиска: добавление, удаление и поиск узла.

Двоичное дерево — структура данных, в которой **каждый узел** (родительский) **имеет не более двух потомков** (правый и левый наследник).

Двоичное дерево поиска строится по определенным правилам:

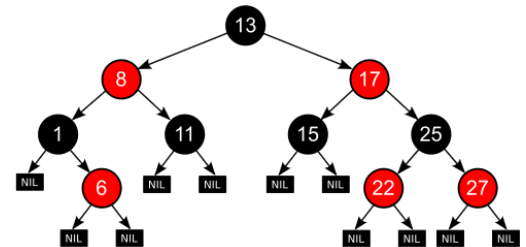
- каждый узел имеет не более двух детей;
- каждое значение, **меньшее, чем значение узла**, становится **левым ребенком** или ребенком левого ребенка;

Расскажите про красно-чёрное дерево.

Протестировать визуализацию дерева → <https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

Каковы особенности красно-чёрного дерева?

- 1) Оба поддерева являются **бинарными деревьями поиска**
- 2) Левые потомки должны быть меньше своего корневого узла (или равны ему), а **правый узел всегда больше левого** (за счёт этого обеспечивается быстрый бинарный поиск)
- 3) Каждый узел окрашен либо в красный, либо в черный цвет (в структуре данных узла появляется доп. поле – бит цвета).
- 4) Корень и листья (так называемые NULL-узлы) окрашены в черный цвет.
- 5) Каждый красный узел должен иметь два черных дочерних узла. Красные узлы в качестве дочерних могут иметь только черные.
- 6) Пути от узла к его листьям должны содержать **одинаковое количество черных узлов** (это черная высота).
- 7) При нарушении этого порядка дерево пере-балансируется.



Расскажите про линейный и бинарный поиск.

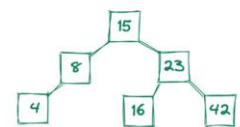
Линейный поиск: сложность алгоритма линейная т.е. зависит от кол-ва элементов.

- Ввод: массив **A**, состоящий из **n** элементов и значение **k**, который мы ищем в массиве
- Вывод: индекс **i**, такой что **A[i] = k**. Если такого **i** нет, то -1. (если элементов **A[i] = k** несколько, то вернуть любой из них, например первый)



- На каждой итерации фиксированное количество операций - **$O(1)$**
- В худшем случае **n** итераций
- **Сложность алгоритма - $O(n)$**

```
public static int linearSearch(int[] A, int k) {
    for (int i = 0; i < A.length; i++) {
        if (A[i] == k)
            return i;
    }
    return -1;
}
```



Расскажите про очередь и стек.

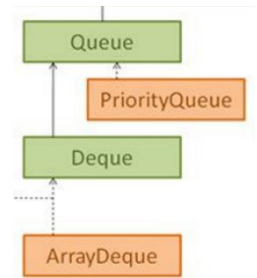
Что такое Queue?

public interface **Queue**<E> extends Collection<E>

Queue – это односторонняя очередь, когда элементы можно получить в том порядке, в котором добавляли. FIFO (первым вошёл, первым вышел).

Согласно Javadoc очереди, очередь добавляет следующие методы:

	Throws exception Выдает исключение	Returns special value Возвращает специальное значение
Insert (вставить)	add(e) – добавь	offer(e) - предложить
Remove (удалить)	remove() - удали	poll() – срезать верхушку
Examine	element() – дай элемент	peek() - посмотреть



Что такое Deque? Чем отличается от Queue? разница между Queue, Deque и Stack?

Deque (Double Ended Queue) – это двусторонняя очередь, т.е. можно вставлять/получать элементы как из начала, так и с конца. Расширяет Queue. Согласно документации, это линейная коллекция, поддерживающая вставку/извлечение элементов с обоих концов (реализация: LIFO, либо FIFO).

Реализации и Deque, и Queue обычно **НЕ переопределяют методы equals() и hashCode()**, вместо этого используются унаследованные методы класса Object, основанные на сравнении ссылок.

Queue – это односторонняя очередь, которая обычно (но необязательно) строится по принципу FIFO (First-In-First-Out). Соответственно извлечение элемента осуществляется с начала очереди, а вставка элемента в конец очереди.

Хотя этот принцип нарушает, к примеру **PriorityQueue**, использующая «natural ordering» или переданный Comparator при вставке нового элемента.

java.util

Interface Deque<E>

Type Parameters:

E - the type of elements held in this collection

All Superinterfaces:

Collection<E>, Iterable<E>, Queue<E>

All Known Subinterfaces:

BlockingDeque<E>

All Known Implementing Classes:

ArrayDeque, ConcurrentLinkedDeque, LinkedBlockingDeque, [LinkedList](#)

Приоритетная очередь

PriorityQueue

- Элементы располагаются в очереди в порядке сравнения: Comparable, natural order, Comparator
- Создайте
 - Queue<Integer> q = LinkedList<>();
 - Заполните элементами от 5 до 1
 - Напечатайте элементы очереди:
 - while (!q.isEmpty()) { q.poll() }
 - Выполните то же самое для PriorityQueue

Приоритетная очередь

Пример

- Создайте очередь чисел, которая сначала возвращает все четные, а потом нечетные числа в порядке возрастания:
 - q.add(5); q.add(2); q.add(1); q.add(4);
 - Poll: 2,4,1,5

Приведите пример реализации Deque.

https://youtu.be/5_f5foEXiYY

Например, класс ArrayDeque<E>.

Этот класс представляют обобщенную двунаправленную очередь, наследуя функционал от класса `AbstractCollection` и применяя интерфейс `Deque`.

В классе `ArrayDeque` определены следующие конструкторы:

- `ArrayDeque()`: создает пустую очередь
- `ArrayDeque(Collection<? extends E> col)`: создает очередь, наполненную элементами из коллекции `col`
- `ArrayDeque(int capacity)`: создает очередь с начальной емкостью `capacity`.
Если мы явно не указываем начальную емкость, то **емкость по умолчанию будет равна 16**

Пример использования класса:

```
// очередь из объектов Person
ArrayDeque<Person> people = new ArrayDeque<Person>();
people.addFirst(new Person("Tom"));
people.addLast(new Person("Nick"));
// перебор без извлечения
for(Person p : people){

    System.out.println(p.getName());
}
```

Какая коллекция реализует FIFO?

FIFO - First-In-First-Out (первый пришел, первым ушел). По этому принципу обычно построена такая структура данных, как очередь (`java.util.Queue`).

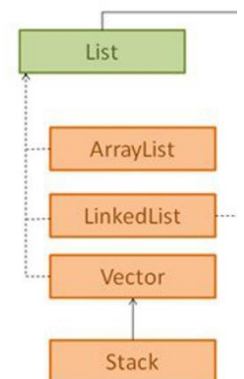
Какая коллекция реализует LIFO?

Stack работает по схеме LIFO (последним вошел, первым вышел, как стопка книг). Всякий раз, когда вызывается новый метод, содержащий примитивные значения или ссылки на объекты, то на вершине стека под них выделяется блок памяти.

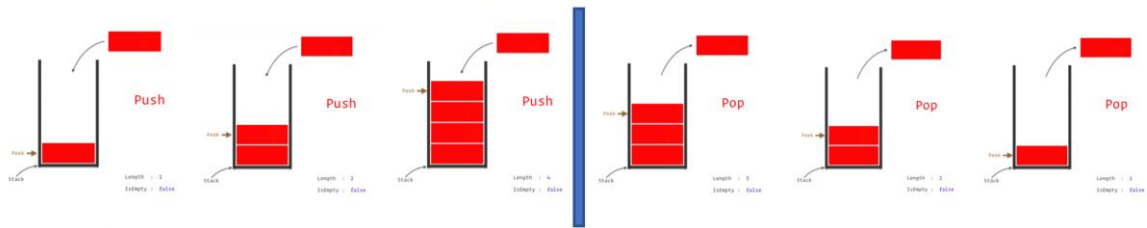
Stack реализует дополнительные методы: `peek` (взглянуть, посмотреть), `pop` (вытолкнуть), `push` (затолкать).

Класс `Vector` является потоком безопасным. Это означает, что, если один поток работает над `Vector`, ни один другой поток не сможет его удерживать. В отличие от `ArrayList`, только один поток может выполнять операцию по вектору за раз. `ArrayList` не синхронизирован, что означает, что в `ArrayList` одновременно могут работать несколько потоков. Таким образом, вы не получите исключение `ConcurrentModificationException`. **Если потоковая реализация не нужна, рекомендуется использовать `ArrayList` вместо `Vector`.**

Значение `Vector` по умолчанию удваивает размер его массива, в то время как `ArrayList` увеличивает его размер массива на 50 процентов. В зависимости от того, как вы используете эти классы, вы можете получить **большой удар производительности при добавлении новых элементов**.



Стек работает по схеме **LIFO** (последним вошел, первым вышел, похоже на стопку книг).



Как это работает:

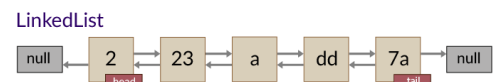
Когда вызывается новый метод, содержащий примитивные значения или ссылки на объекты, то на вершине стека под них выделяется блок памяти. Когда метод завершает выполнение, блок памяти (frame), отведенный для его нужд, очищается, и пространство становится доступным для следующего метода. При этом поток выполнения программы возвращается к месту вызова этого метода с последующим переходом к следующей строке кода.

Сравните сложность вставки, удаления, поиска и доступа по индексу в `ArrayList` и `LinkedList`.

Чем отличаются `ArrayList` и `LinkedList`?

Вопрос проверяет знание особенностей реализации `ArrayList` и `LinkedList` и эффективности операций в этих разных реализациях.

ArrayList vs. LinkedList



Array and ArrayList

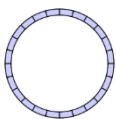


В вопрос иногда добавляют `Vector` – пере-синхронизированный и устаревший вариант `ArrayList`, который лучше заменить `Collections.synchronizedList()`.

`ArrayList` хранит данные в массиве, `LinkedList` в связанном списке. Из этого вытекает разница в эффективности разных операций:

- `ArrayList` лучше справляется с изменениями в середине и ростом в пределах capacity
- `LinkedList` – на краях. **В целом обычно `ArrayList` лучше.**

Стоит добавить, что для работы на краях лучше использовать реализации специально для этого спроектированного интерфейса `Deque`: например, реализующую **кольцевой буфер*** `ArrayDeque`.



***Кольцевой буфер**, или **циклический буфер** — это структура данных, использующая единственный буфер фиксированного размера таким образом, как будто бы после последнего элемента сразу же снова идет первый. Такая структура легко предоставляет возможность буферизации потоков данных.

Скорость основных операций.

LinkedList знает, где находится его голова и где находится хвост)))

ArrayList

addLast - $O[1]$
get - $O[1]$
removeFirst - $O[N]$
insert - $O[N]$
reallocation - $O[N]$

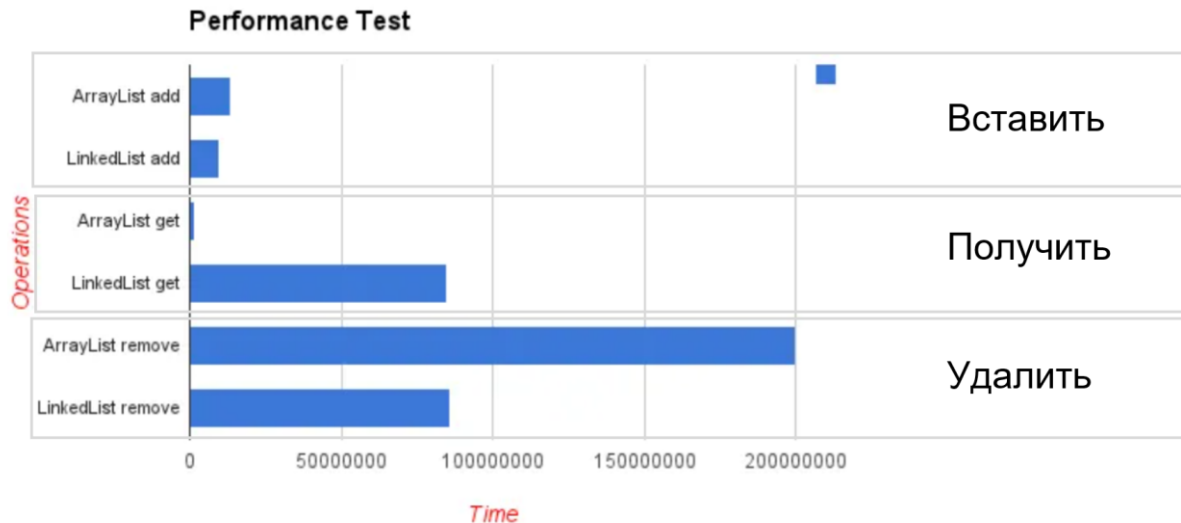
LinkedList

addLast - $O[1]$
get - $O[N]$
removeFirst - $O[1]$
insert - $O[1]$
reallocation - $O[N]$

Remove elements from middle (100k)
LinkedList: 7519 ms
ArrayList: 1544 ms

линкед лист тяжелее (память)
в угоду памяти уменьшить скорость (ArrayList)

	Временная сложность							
	Среднее				Худшее			
	Индекс	Поиск	Вставка	Удаление	Индекс	Поиск	Вставка	Удаление
ArrayList	O(1)	O(n)	O(n)	O(n)	O(1)	O(n)	O(n)	O(n)
LinkedList	O(n)	O(n)	O(1)	O(1)	O(n)	O(n)	O(1)	O(1)



Что следует помнить о **LinkedList**, решая, использовать ли данную коллекцию:

- не синхронизирована
- позволяет хранить любые объекты, в том числе null и повторяющиеся
- за **константное время O(1)** выполняются операции вставки и удаления первого и последнего элемента и операции вставки и удаления элемента из середины списка (**не учитывая время поиска позиции** элемента, который осуществляется за линейное время)
- за **линейное время O(n)** выполняются операции поиска элемента по индексу и по значению

ArrayList – это **динамический массив**, т.е. может менять свой размер во время исполнения программы, при этом не обязательно указывать размерность при создании объекта. Элементы ArrayList могут быть абсолютно любых типов в том числе и null.

Используем тогда, когда нам нужна структура, похожая на массив, но где нам нужно добавлять/удалять/изменять элементы. Получение и изменение элементов **выполняется быстро**, поскольку эти операции просто обращаются к соответствующему элементу массива». В основе ArrayList лежит массив Object (элементами явл. Объекты типа Object).

Ёмкость **capacity** массива по дефолту – **10 мест** (не путать размер и ёмкость).

Размер массива – это сколько по факту лежит элементов в массиве, а ёмкость – это потенциально возможное кол-во мест).

Создание:

```

1) ArrayList<DataType> list1 = new ArrayList<DataType>();
2) ArrayList<DataType> list1 = new ArrayList<>(); // можем не писать повторно

Ёмкость capacity (по дефолту = 10 мест)
Если ёмкость знаем заранее, то нужно это явно указать
3) ArrayList<DataType> list2 = new ArrayList<>(int capacity: 200); // сделали так - увеличили скорость добавления элементов
Это размер базового массива, который используется для хранения элементов.
Ёмкость растёт автоматически по мере добавления элементов в список массивов.
Ёмкость - это размер базового массива, который используется для хранения элементов.

НЕ ПУТАТЬ: size (это размер, т.е. сколько фактически лежит внутри элементов) и capacity (ёмкость, т.е. сколько мест выделено под заполнение)
4) ArrayList<DataType> list4 = new ArrayList<>(list3); // в параметры конструктора помещаем другой AL // ЭТО НОВЫЙ ОБЪЕКТ

```

Конструктор и описание

```

ArrayList() конструктор создаёт пустой список массивов.
ArrayList(Collection c) конструктор строит список массивов, кот. иници-ся элем. коллекции c.
ArrayList(int capacity) конструктор создаёт список массивов с указанной начальной capacity.

```

Скорость основных операций

	Временная сложность							
	Среднее				Худшее			
	Индекс	Поиск	Вставка	Удаление	Индекс	Поиск	Вставка	Удаление
ArrayList	O(1)	O(n)	O(n)	O(n)	O(1)	O(n)	O(n)	O(n)

- Быстрый доступ к элементам по индексу за константное время $O(1)$
- Доступ к элементам по значению за линейное время $O(n)$
- Медленный, когда вставляются и удаляются элементы из «середины» списка
- Позволяет хранить любые значения в том числе и null
- Не синхронизирован

Есть понятие Время доступа (O большая). В LinkedList (связанный список) идёт перебор элементов, поэтому при вставке в конец мы сначала проходим весь список (линейное время), а потом добавляем массив. Линейное время. А ArrayList - это динамический массив (доступ к элементам по индексу). Поиск значения $O(n)$, а в ArrayList $O(1)$ то есть константное время в любую точку массива вставка.

ОТВЕТ: в середину и в конец динам. массива ArrayList БЫСТРЕЕ ВСТАВКА (еще нативный метод используется `System.arraycopy()`);

А LinkedList используется редко, т.к. он много места в памяти занимает и подходит для небольшого кол-ва данных.

ВОТ СЛУЧАИ, ГДЕ LinkedList БЫСТРЕЕ, в остальном ArrayList - чемпион!

---Insert elements to begin(100k) ---/--- Add elements (6kk) _/ ___ Remove elements from begin (100k) _/ ___ Set elements at end (3kk)

LinkedList: 132 ms _____ / _ LinkedList: 2264 ms _ / _ LinkedList: 2 ms _____ / _ LinkedList: 40 ms

ArrayList: 2742 ms _____ / _ ArrayList: 493 ms _ / _ ArrayList: 3220 ms _____ / _ ArrayList: 267 ms

LinkedList is faster _____ / _ **ArrayList is faster** _____ / _ LinkedList is faster _____ / _ LinkedList faster

Алгоритм основных операций <https://habr.com/ru/post/128269/>

Если вызывается конструктор без параметров, то по умолчанию будет создан массив из 10-ти элементов типа Object (с приведением к типу, разумеется), индексы от 0 до 9.

```
elementData = (E[]) new Object[10];
```

Можно использовать конструктор `ArrayList(capacity)` и указать свою начальную емкость.

1) **Добавление элементов** `list.add("0");`

Внутри метода `add(value)` происходят следующие вещи:

- проверяется, достаточно ли места в массиве для вставки нового элемента `ensureCapacity(size + 1);`
- добавляется элемент в конец (согласно значению `size`) массива `elementData[size++] = element;`

Если места в массиве недостаточно, новая емкость рассчитывается по формуле $(oldCapacity * 3) / 2 + 1$.

Второй момент — это копирование элементов. Оно осуществляется с помощью нативного метода `System.arraycopy()`, который написан не на языке Java.

```
// newCapacity - новое значение емкости
elementData = (E[])new Object[newCapacity];
```

```
// oldData - временное хранилище текущего массива с данными
System.arraycopy(oldData, 0, elementData, 0, size);
```

При добавлении 11-го элемента, проверка показывает что места в массиве нет. Соответственно создается новый массив и вызывается `System.arraycopy()`.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
"0"	"1"	"2"	"3"	"4"	"5"	"6"	"7"	"8"	"9"	null	null	null	null	null	null

После этого добавление элементов продолжается

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
"0"	"1"	"2"	"3"	"4"	"5"	"6"	"7"	"8"	"9"	"10"	null	null	null	null	null

2) Добавление в «середину» списка `list.add(5, "100");`

Добавление элемента на позицию с определенным индексом происходит в три этапа:

- проверяется, достаточно ли места в массиве для вставки нового элемента
`ensureCapacity(size+1);`
- подготавливается место для нового элемента с помощью **`System.arraycopy()`**;
`System.arraycopy(elementData, index, elementData, index + 1, size - index);`
- перезаписывается значение у элемента с указанным индексом
`elementData[index] = element;`
`size++;`

В случаях, когда происходит вставка элемента по индексу и при этом в вашем массиве нет свободных мест, то вызов **`System.arraycopy()`** случится дважды: первый в **`ensureCapacity()`**, второй в самом методе **`add(index, value)`**, что явно скажется на скорости всей операции добавления.

В случаях, когда в исходный список необходимо добавить другую коллекцию, да еще и в «середину», стоит использовать метод **`addAll(index, Collection)`**. И хотя, данный метод скорее всего вызовет **`System.arraycopy()`** три раза, в итоге это будет гораздо быстрее поэлементного добавления.

3) Удаление элементов

Удалять элементы можно двумя способами:

- по индексу **`remove(index)`**
- по значению **`remove(value)`**

С удалением элемента по индексу всё достаточно просто: `list.remove(5);`

- Сначала определяется, какое количество элементов надо скопировать
`int numMoved = size - index - 1;`
- затем копируем элементы используя **`System.arraycopy()`**
`System.arraycopy(elementData, index + 1, elementData, index, numMoved);`
- уменьшаем размер массива и забываем про последний элемент
`elementData[--size] = null; // Let gc do its work`

При удалении по значению, в цикле просматриваются все элементы списка, до тех пор, пока не будет найдено соответствие. **Удален будет лишь первый найденный элемент.**

При удалении элементов текущая величина `capacity` не уменьшается, что может привести к своеобразным утечкам памяти. Поэтому не стоит пренебрегать методом **`trimToSize()`**.